PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería

PROPUESTA DE PROYECTO DE GRADO

GammaLab 2.0

Análisis y procesamiento
de Señales Electroencefalográficas

Una herramienta de código abierto para investigadores
en análisis de señales neurofisiológicas

ESTUDIANTES

Juan Felipe Romero Ortiz

Laura Juliana Garzón Arias

Xiomara Herrera Suárez

Sebastian Almanza Galvis

DIRECTORA

Cecile Eugenie Gloria Gauthier Umaña

INFORMACIÓN DEL PROYECTO

Objetivo: Extender las capacidades de análisis de señales EEG e incorporar una arquitectura escalable y mejora de interfaz

Bogotá, D.C. — Año 2026

Índice

1. Introducción **3**

2. Vista General del Proyecto **4**

2.1. Visión del Producto 4

2.1.1. Características clave de la visión 4

2.2. Propósito, Alcance y Objetivos 4

2.2.1. Propósito 4

2.2.2. Alcance 5

2.2.3. Objetivo General 5

2.2.4. Objetivos Específicos 6

2.3. Supuestos y Restricciones 6

2.3.1. Supuestos 6

2.3.2. Restricciones 6

2.4. Resumen del Proyecto 7

3. Glosario **7**

4. Contexto del Proyecto **8**

4.1. Modelo de Ciclo de Vida 8

4.2. Proceso de Selección del Modelo de Ciclo de Vida 10

4.3. Lenguajes y Herramientas 13

4.4. Organización del proyecto y Comunicación 14

4.5. Entregables del Proyecto 15

4.6. Estructura colaborativa y enfoque de trabajo 17

5. Administración del Proyecto **18**

5.1. Métodos y Herramientas de Estimación 18

5.1.1. Supuestos de Estimación 18

5.1.2. Estimación de Tiempo por Fases del Proyecto 19

5.1.3. Distribución de Esfuerzo por Fase 19

5.2. Inicio del Proyecto 20

5.2.1. Entrenamiento del Personal 20
5.2.2. Infraestructura 21
5.3. Asignación de Recursos 22
5.3.1. Equipo de Desarrollo 22
5.3.2. Disponibilidad Horaria por Desarrollador 23
5.3.3. Directora del Proyecto 24
5.3.4. Cliente 24
5.4. Gestión de Cambios y Control de Alcance 25
5.4.1. Solicitudes de Cambio 25
5.5. Criterios de Éxito del Proyecto 25
5.6. Monitoreo y Métricas de Avance 26
5.6.1. Métrica Clave: Velocidad del Equipo 26
5.6.2. Reuniones de Sincronización 26

6. Monitoreo y Control del Proyecto **28**
6.1. Administración de Requerimientos 28
6.2. Monitoreo y Control de Progreso 28
6.3. Cierre del Proyecto 30
6.4. Entrega del Producto 30

PMP

GammaLab 2.0

May 2026

1. Introducción

El presente documento constituye el Plan de Administración de Proyecto (PMP) de **GammaLab 2.0**, la versión extendida de la plataforma de código abierto para el análisis y visualización de señales electroencefalográficas (EEG) desarrollada originalmente como GammaLab 1.0. Su propósito es documentar de forma estructurada la planificación, la organización y los mecanismos de control que rigen el desarrollo del proyecto, permitiendo al lector comprender la justificación, el alcance y la estrategia metodológica adoptada para su ejecución.

GammaLab 2.0 surge como respuesta a las brechas técnicas y funcionales identificadas en la primera versión del sistema, con el propósito de ampliar sus capacidades de análisis, fortalecer su arquitectura y mejorar la experiencia de usuario, manteniendo la filosofía de software libre que ha caracterizado al proyecto desde su origen.

Este documento expone de forma estructurada la vista general del proyecto, el contexto metodológico y tecnológico adoptado, los mecanismos de administración y control, y los procesos de soporte que acompañan el desarrollo. La metodología adoptada combina un enfoque secuencial por fases (*Water-Scrum-Fall*) con ciclos iterativos de trabajo al interior de cada una, permitiendo mantener trazabilidad sobre las dependencias técnicas del proyecto sin renunciar a la flexibilidad frente a cambios.

El documento está dirigido a los integrantes del equipo de desarrollo, a la directora del proyecto, a los revisores académicos del trabajo de grado y al Laboratorio de Electrofisiología de la Universidad Nacional de Colombia, como cliente y usuario final del sistema. Su lectura es especialmente útil para comprender la planificación formal y estratégica detrás del desarrollo de GammaLab 2.0, así como los criterios bajo los cuales se medirá su éxito.

2. Vista General del Proyecto

2.1. Visión del Producto

GammaLab 2.0 es una versión extendida de la aplicación GammaLab 1.0 diseñada para el análisis y posprocesamiento de señales electroencefalográficas (EEG). La visión busca democratizar el acceso a herramientas avanzadas de análisis neurofisiológico eliminando barreras técnicas y económicas.

2.1.1. Características clave de la visión

- **Accesibilidad**: Herramienta completamente gratuita y de código abierto (licencia MIT)
- **Facilidad de uso**: Interfaz gráfica intuitiva que no requiere conocimientos avanzados de programación
- **Versatilidad analítica**: Análisis en dominios de tiempo, frecuencia y tiempo-frecuencia
- **Escalabilidad**: Arquitectura microkernel que permite extensiones futuras
- **Impacto científico**: Soporte a investigadores del Laboratorio de Electrofisiología de la Universidad Nacional

2.2. Propósito, Alcance y Objetivos

2.2.1. Propósito

Expandir las capacidades de GammaLab 1.0 mediante nuevas funcionalidades, optimización del rendimiento y corrección de limitaciones identificadas en su primera iteración.

El propósito atiende a dos actores con necesidades distintas, que conviene no confundir:

- **El investigador**, usuario final del sistema: debe poder realizar análisis avanzados de señales EEG **sin experiencia en programación**. Su único punto de contacto con el sistema es la interfaz gráfica, desde la cual se ejecutan todas las operaciones de carga, análisis y visualización.
- **El desarrollador**, responsable de extender el sistema en el futuro: debe poder incorporar nuevos análisis como módulos independientes sobre la arquitectura microkernel, sin modificar el núcleo. Esta extensibilidad **sí exige conocimientos de programación** y queda fuera del trabajo del investigador.

La distinción importa porque la facilidad de uso y la escalabilidad son atributos dirigidos a públicos diferentes, y se evalúan de forma distinta: el primero, en la experiencia del investigador frente a la interfaz; el segundo, en el esfuerzo que le representa a un programador añadir una funcionalidad nueva.

2.2.2. Alcance

El alcance del proyecto GammaLab 2.0 comprende los siguientes elementos, enumerados en el orden en que se abordaron durante la ejecución:

1. **Rediseño de la interfaz gráfica y la experiencia de usuario**: reorganización de la navegación, unificación visual de los paneles de parámetros y de las vistas de resultados.
2. **Corrección de errores y limitaciones de la versión 1.0**: subsanación de los defectos priorizados tras el análisis de brechas de la Fase 1, junto con los detectados durante el propio desarrollo.
3. **Paralelización del procesamiento**: incorporación de un orquestador de tareas que ejecuta los cálculos fuera del hilo de la interfaz, con informe de avance y cancelación, de modo que la aplicación no se bloquee durante los análisis prolongados.
4. **Integración de nuevos métodos de análisis de señales**: implementación del acoplamiento fase-amplitud (*phase-amplitude coupling*) en sus modalidades PAC, PAC Average y PAC Psel, junto con el índice de modulación.
5. **Documentación completa y manual de usuario**: elaborada de forma transversal a lo largo del proyecto y consolidada en el cierre.

El orden no es arbitrario. Los nuevos métodos de análisis se abordaron en último lugar porque dependen de la infraestructura de ejecución asíncrona construida en el punto anterior: así nacieron ya integrados al orquestador, en lugar de requerir una migración posterior.

2.2.3. Objetivo General

Desarrollar GammaLab 2.0 como una versión extendida que amplíe las capacidades de análisis de señales EEG e incorpore una arquitectura escalable, mejorando el rendimiento del sistema y soportando múltiples análisis simultáneamente.

2.2.4. Objetivos Específicos

Los objetivos específicos que guían el desarrollo del proyecto son:

1. Identificar brechas de mejora en la versión 1.0 de GammaLab
2. Diseñar el planteamiento estructural de la solución para abordar las brechas detectadas
3. Implementar la solución técnica completa a través de software
4. Validar la solución mediante pruebas funcionales y de rendimiento con usuarios finales

2.3. Supuestos y Restricciones

2.3.1. Supuestos

El proyecto se fundamenta en los siguientes supuestos:

- La institución (Universidad Nacional de Colombia) proporcionará acceso a bases de datos de señales EEG para validación y pruebas
- El equipo de desarrollo tendrá disponibilidad de tiempo dedicado durante el período del proyecto
- El equipo de GammaLab 1.0 facilitará el acceso al material original del proyecto —código fuente, documentación técnica y de gestión, el programa en MATLAB (*BOARD_FTD_PACC*) que sirvió de referencia a la versión 1.0, y las señales EEG empleadas en su validación— y atenderá consultas sobre las decisiones de diseño que no quedaron documentadas. GammaLab 2.0 parte de ese material como línea base
- Los investigadores del laboratorio participarán activamente en pruebas, validación y retroalimentación del sistema
- Los estándares de calidad establecidos (ISO/IEC, IEEE, PMI) serán aplicables al contexto académico del proyecto

2.3.2. Restricciones

El proyecto opera bajo las siguientes restricciones:

- **Restricciones Temporales**: El proyecto debe completarse dentro del cronograma académico de grado establecido por la institución

- **Restricciones Técnicas**: Dependencia de Python y librerías de código abierto (NumPy, SciPy, MNE-Python, PyQt/PySide) para el desarrollo
- **Restricciones de Recursos**: Disponibilidad limitada de equipamiento especializado en el laboratorio para pruebas de rendimiento y validación
- **Restricciones de Conocimiento**: Limitación inicial en la comprensión profunda del código heredado de GammaLab 1.0, requiriendo fase de exploración técnica
- **Restricciones de Compatibilidad**: Necesidad de mantener retrocompatibilidad donde sea posible con funcionalidades existentes de la versión 1.0
- **Restricciones de Equipo**: Equipo conformado por 4 estudiantes con roles de desarrollo colaborativo bajo metodología ágil

2.4. Resumen del Proyecto

La siguiente tabla sintetiza las características principales del proyecto GammaLab 2.0:

Aspecto	**Descripción**
Producto	GammaLab 2.0 - Software de análisis de señales EEG
Usuarios Finales	Investigadores del Laboratorio de Electrofisiología (UNC)
Licencia	MIT (Open Source)
Costo	Gratuito
Diferenciador	Única herramienta gratuita con GUI intuitiva sin requerimientos de programación
Metodología	Water-Scrum-Fall (4 fases, Scrum iterativo dentro de cada fase)
Equipo	4 estudiantes + 1 directora (Product Owner)
Duración	Proyecto académico de grado

Cuadro 1: Resumen de características principales del proyecto GammaLab 2.0

3. Glosario

Cuadro 2: Acrónimos y términos utilizados en el documento

Sigla / Término	**Definición**
SAD	*Software Architecture Document*. Documento de arquitectura del sistema.

Sigla / Término	Definición
SRS	*Software Requirements Specification*. Documento de especificación de requisitos.
SDD	*Software Design Document*. Documento de diseño técnico.
IoC	Inversión de Control. Principio que delega el flujo de ejecución a un contenedor.
DI	*Dependency Injection*. Técnica para resolver dependencias desde un contenedor externo.
MVP / MVC	Patrones de separación de interfaz y lógica (*Model–View–Presenter* / *Model–View–Controller*).
Template Method	Patrón que define un flujo general con pasos sobrescribibles.
Adapter	Patrón para estandarizar interfaces incompatibles.
Caching	Estrategia para evitar recalcular o recomponer estructuras de datos.
Python	Lenguaje de programación principal de *Gamma Lab*.
PyQt5	Framework de interfaz gráfica basado en Qt para Python.
VTK	*Visualization Toolkit*. Biblioteca para visualización científica 2D/3D.
QVTK	Interfaz `QVTKRenderWindowInteractor` para incrustar VTK en aplicaciones Qt.
NumPy	Librería para cálculo numérico vectorizado.
EDF	*European Data Format*. Formato estándar para señales biomédicas.
ABF	*Axon Binary File*. Formato de archivo utilizado por Axon Instruments para registros biomédicos.
GUI Toolkit	Conjunto de herramientas para construir interfaces gráficas (Qt, PyQt, VTK).

4. Contexto del Proyecto

4.1. Modelo de Ciclo de Vida

GammaLab 2.0 adopta un ciclo de vida híbrido tipo *Water-Scrum-Fall*, en el cual la estructura global del proyecto sigue una lógica secuencial, mientras que la ejecución interna de cada etapa se desarrolla bajo dinámicas iterativas inspiradas en Scrum.

La naturaleza secuencial del modelo se refleja en la organización del proyecto alrededor de sus cuatro objetivos específicos, los cuales establecen una dependencia lógica entre etapas. En este sentido, el avance del proyecto ocurre mediante una progresión ordenada donde cada fase produce los insumos necesarios para habilitar la siguiente, conservando así una trazabilidad clara del desarrollo y una planificación estructurada del proyecto. Las fases son las siguientes:

Fase 1: Identificación de brechas de mejora en GammaLab 1.0. Se analiza el proyecto antecesor desde perspectivas técnicas y funcionales para

identificar oportunidades de mejora y definir prioridades para la nueva versión.

Fase 2: Diseño del planteamiento estructural de la solución. Se define la arquitectura general, el planteamiento estructural y las propuestas de mejora funcional y de experiencia de usuario para GammaLab 2.0.

Fase 3: Implementación técnica. Se desarrollan e integran de manera incremental las funcionalidades priorizadas y los nuevos componentes definidos en la etapa de diseño.

Fase 4: Validación y pruebas con el usuario. Se evalúa el desempeño técnico y funcional de GammaLab 2.0 mediante pruebas comparativas y validación con usuarios.

Adicionalmente, al interior de cada una de estas fases el trabajo se organiza de forma iterativa mediante ciclos cortos de desarrollo (*sprints*), permitiendo distribuir tareas, realizar seguimiento continuo al avance e incorporar retroalimentación de manera progresiva. Esta dinámica favorece la adaptación frente a ajustes funcionales, refinamiento de requerimientos o decisiones de diseño que emerjan durante el proceso.

Aunque el flujo principal mantiene una dirección secuencial, el modelo contempla retroalimentación entre fases cuando se identifican cambios relevantes. En estos casos, es posible retornar a una etapa previa para realizar ajustes específicos; por ejemplo, desde implementación hacia diseño o desde validación hacia implementación. Sin embargo, dichos retornos no rompen la lógica general del ciclo de vida: una vez realizado el ajuste correspondiente, el proyecto debe continuar nuevamente de manera secuencial a través de las fases subsiguientes. De esta forma, el modelo combina flexibilidad operativa con control estructural sobre el avance global del proyecto.

En términos organizacionales, el trabajo iterativo se adapta a un equipo de cuatro integrantes con rol de desarrolladores. Adicionalmente, uno de los integrantes asume funciones de coordinación equivalentes al *Scrum Master*, enfocadas en la planeación y seguimiento del trabajo, mientras que la directora del proyecto cumple un rol análogo al *Product Owner*, centralizando la retroalimentación del cliente y la validación de avances.

Bajo esta configuración, el ciclo de vida de GammaLab 2.0 integra planificación por fases, iteración continua y mecanismos controlados de retroalimentación entre etapas, alineándose tanto con las dependencias técnicas del proyecto como con la necesidad de adaptación durante el desarrollo. Todo lo anterior se representa en el diagrama del ciclo de vida metodológico de GammaLab 2.0, presentado en la Figura 1.

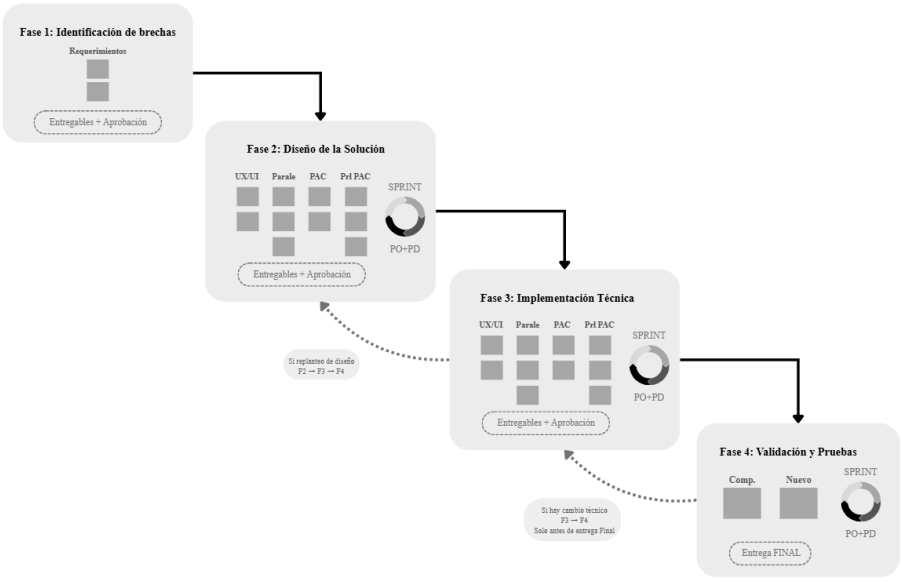


Figura 1: Ciclo de vida metodológico adoptado para el desarrollo de GammaLab 2.0 bajo enfoque Water-Scrum-Fall.

4.2. Proceso de Selección del Modelo de Ciclo de Vida

El desarrollo de un sistema de software requiere, antes de cualquier decisión técnica, definir cómo se organizará el trabajo a lo largo del tiempo. Esta organización se formaliza a través de un modelo de ciclo de vida del software, que Pressman & Maxim definen como una representación abstracta de la secuencia de actividades que guían al equipo desde la recolección de requisitos hasta la entrega y el mantenimiento del sistema [1].

En términos generales, los modelos de ciclo de vida se agrupan en dos grandes categorías: los modelos dirigidos por plan (plan-driven), donde todas las fases se planifican en detalle desde el inicio y el progreso se mide contra ese plan; y los modelos ágiles, donde la planificación es incremental y el proceso se adapta continuamente a los cambios del entorno y de los requisitos [3]. Entre estos dos extremos existe un espectro de variantes que difieren en aspectos como la secuencialidad de sus fases, el nivel de documentación exigido, la frecuencia de entrega de resultados y el grado de participación del cliente durante el desarrollo. A continuación se presenta una revisión de los modelos de mayor relevancia en la literatura, con el propósito de fundamentar la elección metodológica para el desarrollo de GammaLab 2.0.:

1. **Modelo en cascada.** Propone una estrategia de organización estrictamente secuencial, en donde se propone el siguiente orden para los pro-

yectos: requisitos, diseño, implementación, pruebas y mantenimiento. Se caracteriza porque en cada fase debe completarse y producir su documentación formal antes de iniciar la siguiente. Los recursos se asignan fase por fase, lo que típicamente implica equipos especializados por etapa con poca interacción cruzada. Es apropiado para proyectos grandes, con requisitos bien definidos desde el inicio, equipos distribuidos geográficamente y donde la documentación formal es un requisito contractual [1].

2. **Modelo en V.** Se presenta como una extensión directa del modelo en cascada [1]. Su diferencia central es que a cada fase de desarrollo le corresponde simétricamente una fase de verificación o prueba: los requisitos se asocian a pruebas de aceptación, el diseño del sistema a pruebas de integración, y el diseño detallado a pruebas unitarias. Los entregables y la documentación se generan en ambos lados de la V, lo que hace que el plan de pruebas se diseñe en paralelo al desarrollo desde las fases más tempranas. Al igual que la cascada, es un modelo rígido y secuencial, con equipos especializados por fase [5].

3. **Modelo incremental.** Combina elementos del modelo en cascada aplicados de forma iterativa y escalonada [1]. El sistema se divide en incrementos funcionales que se desarrollan y entregan en secuencia; el primer incremento cubre la funcionalidad más crítica y cada entrega posterior añade capacidades adicionales. A diferencia de la cascada, los recursos se distribuyen de forma continua a lo largo del proyecto y los equipos son multifuncionales, trabajando especificación, diseño e implementación para cada incremento. La comunicación con el cliente es más frecuente al haber entregas periódicas [3].

4. **Modelo en espiral.** Se diferencia de los anteriores por centrar el proceso en la gestión del riesgo [1]. El proyecto avanza en iteraciones cíclicas, cada vuelta de la espiral pasa por planificación, análisis de riesgos, desarrollo y evaluación y en cada ciclo el sistema se hace más completo. Los recursos se consumen de manera incremental por ciclo, lo que permite ajustar el alcance según los riesgos identificados. La documentación y los entregables se producen al final de cada iteración. Es apropiado para proyectos grandes, complejos, de larga duración y con alta incertidumbre técnica [3].

5. **Scrum.** Es un marco ágil que organiza el trabajo en ciclos cortos de duración fija llamados sprints, típicamente de dos a cuatro semanas, al final de los cuales se entrega un incremento funcional del producto [2]. Define tres roles claros: el Product Owner, que prioriza el trabajo; el Scrum Master, que facilita el proceso; y el equipo de desarrollo, que es pequeño, multifuncional y autogestionado. La documentación es ligera y los entregables son incrementos de software funcional evaluados en cada Sprint Review. La comunicación es intensiva y diaria dentro del equipo. Es recomendado para equipos pequeños con requisitos cambiantes y acceso continuo al cliente [1].

Con base en la revisión anterior, la siguiente tabla sintetiza las características más relevantes de cada modelo para efectos de la selección metodológica:

Criterio	Cascada	Modelo V	Incremental	Espiral	Scrum
Documentación	Extensa y formal por fase	Extensa, incluye plan de pruebas desde el inicio	Moderada por incremento	Moderada por iteración	Ligera, orientada a entregables funcionales
Tamaño del proyecto	Grande	Grande	Mediano	Grande y complejo	Pequeño a mediano
Número de integrantes	Equipos grandes y especializados	Equipos grandes y especializados	Equipos medianos multifuncionales	Equipos medianos con perfil en gestión de riesgos	Equipos pequeños (3–9 personas)
Interacción con el cliente	Baja, concentrada al inicio y al final	Baja, concentrada al inicio y al final	Media, en cada entrega incremental	Media, al cierre de cada iteración	Alta, continua durante los sprints
Flexibilidad ante cambios	Muy baja	Muy baja	Media	Alta por gestión de riesgos	Muy alta
Testeo	Al final del desarrollo	Planificado desde el inicio, paralelo al diseño	Al final de cada incremento	Al final de cada iteración	Continuo dentro de cada sprint
Entrega de valor	Única, al final del proyecto	Única, al final del proyecto	Parcial y progresiva	Parcial por iteración	Parcial al cierre de cada sprint

Cuadro 3: Comparación de modelos de ciclo de vida

Al contrastar estas características con el contexto de GammaLab 2.0, se observa que ningún modelo satisface por sí solo las necesidades del proyecto. Por una parte, existen dependencias técnicas claramente secuenciales derivadas de los objetivos específicos: no es posible implementar sin antes diseñar la arquitectura, ni diseñar sin haber identificado y priorizado previamente las brechas. Esto evidencia la necesidad de una estructura por fases y cierto nivel de planificación anticipada.

Sin embargo, el proyecto también presenta condiciones que demandan flexibilidad. El equipo está conformado por cuatro integrantes, los requisitos funcionales continúan definiéndose durante el desarrollo y la interacción con el cliente es frecuente, con retroalimentación constante de la directora del proyecto. Estas características resultan incompatibles con la rigidez documental y la baja capacidad de adaptación de modelos como cascada y modelo en V.

Por otra parte, metodologías puramente ágiles como Scrum se ajustan mejor al tamaño del equipo y a la dinámica iterativa del trabajo, pero no proporcionan por sí solas una estructura suficientemente robusta para gestionar las dependencias entre las grandes etapas del proyecto.

Sommerville (2016) señala que, en la práctica, los procesos de software reales rara vez adoptan un único modelo de manera pura, sino que combinan elementos de enfoques dirigidos por plan y enfoques ágiles de acuerdo con las características y restricciones particulares de cada proyecto. Yahya & Maidin (2022) documentan empíricamente esta tendencia, observando que numerosos proyectos

recurren a modelos híbridos que combinan la estructura secuencial de metodologías tradicionales con la flexibilidad iterativa de Scrum, configurando lo que se conoce como *Water-Scrum-Fall*. Este enfoque conserva una organización global por fases, permitiendo gestionar dependencias técnicas y mantener trazabilidad sobre el avance del proyecto, mientras incorpora dinámicas ágiles al interior de cada etapa mediante trabajo iterativo, entregables parciales y retroalimentación continua [4].

Por estas razones, y en coherencia con las necesidades previamente identificadas para GammaLab 2.0, se selecciona Water-Scrum-Fall como el ciclo de vida metodológico más adecuado para el desarrollo del proyecto.

4.3. Lenguajes y Herramientas

Dado que GammaLab 2.0 corresponde a una evolución directa de GammaLab 1.0, el desarrollo técnico del proyecto prioriza la continuidad tecnológica y la reutilización de la arquitectura existente. Esta decisión responde al propósito de preservar el trabajo previamente realizado, mantener compatibilidad con los módulos implementados y reducir costos asociados a una reestructuración completa del sistema.

En este sentido, la mayoría de los lenguajes y herramientas empleados en GammaLab 1.0 se conservan para esta nueva versión. No obstante, GammaLab 2.0 incorpora mejoras orientadas a solucionar limitaciones identificadas en la versión anterior, particularmente relacionadas con estabilidad técnica, experiencia de usuario y extensión de funcionalidades para análisis de señales EEG.

La Tabla 4 resume el conjunto de tecnologías utilizadas en el desarrollo del proyecto.

Categoría	**Tecnologías empleadas**
Lenguaje de programación	Python
Desarrollo de interfaz gráfica	PySide6, Qt Designer
Entorno de desarrollo	Visual Studio Code
Control de versiones	GitHub
Gestión del proyecto	Tablero Kanban (proporcionado por el departamento)
Documentación técnica	LaTex, Microsoft Word
Pruebas	Pytest
Diseño de interfaz y diagramas	Figma, Draw.io, Canva
Comunicación y colaboración	Microsoft Teams, WhatsApp
Almacenamiento y respaldo	OneDrive, máquina virtual compartida

Cuadro 4: Lenguajes y herramientas empleadas en GammaLab 2.0

Python se mantiene como lenguaje principal del proyecto, dado que Gam-

maLab fue originalmente desarrollado en este entorno y cuenta con compatibilidad directa con bibliotecas científicas ampliamente utilizadas en procesamiento de señales, como NumPy, SciPy, MNE-Python y PyEEG. Esto permite conservar la base existente e incorporar nuevas funcionalidades de análisis EEG sin modificar el ecosistema tecnológico principal.

Para la interfaz gráfica se conserva el uso de **PySide6** y **Qt Designer**, herramientas que permiten mantener coherencia con la arquitectura visual existente y facilitan la separación entre diseño de interfaz y lógica funcional.

En términos de desarrollo colaborativo, el proyecto utiliza **GitHub** para control de versiones, gestión de ramas y revisión de cambios, complementados con el **tablero Kanban proporcionado por el departamento** como herramienta de seguimiento de tareas acorde con la dinámica iterativa definida en la metodología del proyecto.

La documentación técnica y académica se desarrolla principalmente en **LaTeX**, favoreciendo trazabilidad, edición colaborativa y consistencia de formato. Adicionalmente, se emplea **Microsoft Word** para documentación preliminar y trabajo colaborativo informal.

Para diseño y modelado visual se utilizan **Figma** en prototipado de interfaces, **Draw.io** para diagramas técnicos y **Canva** para material gráfico de apoyo.

Finalmente, las pruebas de calidad se incorporan mediante **Pytest**, mientras que la coordinación del equipo se apoya en **Microsoft Teams** y **WhatsApp**. Como mecanismo de respaldo y pruebas compartidas, el proyecto cuenta adicionalmente con almacenamiento en **OneDrive** y una máquina virtual compartida.

4.4. Organización del proyecto y Comunicación

La organización de GammaLab 2.0 responde a la naturaleza colaborativa e iterativa del proyecto, integrando una estructura ligera de coordinación alineada con el enfoque metodológico Water-Scrum-Fall adoptado.

A nivel organizacional, el cliente del proyecto corresponde al Laboratorio de Electrofisiología de la Universidad Nacional de Colombia, entidad beneficiaria del desarrollo y principal fuente de requerimientos funcionales y criterios de validación del sistema.

La directora del proyecto actua como puente de comunicación entre el equipo de desarrollo y el cliente. Sus responsabilidades incluyen facilitar la coordinación de reuniones con el laboratorio, orientar la toma de decisiones relacionadas con alcance, complejidad técnica y priorización funcional, así como ejercer supervisión técnica y académica sobre el desarrollo del proyecto.

El equipo de desarrollo está conformado por cuatro integrantes, todos con participación activa en las distintas fases del proyecto. La distribución del trabajo se realiza de manera colaborativa y equitativa, asignando responsabilidades

de acuerdo con los frentes funcionales definidos y promoviendo revisión cruzada entre integrantes para asegurar consistencia técnica e integración del sistema.

Adicionalmente, uno de los integrantes asume funciones equivalentes al rol de *Scrum Master*, en calidad de líder del equipo. Este rol se enfoca en la coordinación general del trabajo, seguimiento del cronograma, organización de reuniones y monitoreo del avance del proyecto, sin implicar una carga adicional de desarrollo respecto a los demás integrantes. La ejecución técnica y cumplimiento de objetivos se mantiene distribuida de manera equitativa entre todos los miembros.

La comunicación formal del proyecto se realiza mediante reuniones periódicas con la directora, orientadas al seguimiento metodológico, revisión de avances y resolución de decisiones técnicas o estratégicas. De forma complementaria, el equipo mantiene canales de comunicación interna para coordinación diaria, seguimiento de tareas y gestión ágil de bloqueos operativos.

Esta estructura organizacional permite mantener claridad en la toma de decisiones, supervisión continua y comunicación efectiva con el cliente, preservando al mismo tiempo una dinámica horizontal y colaborativa dentro del equipo de desarrollo.

La estructura organizacional del proyecto se presenta en la Figura 2.

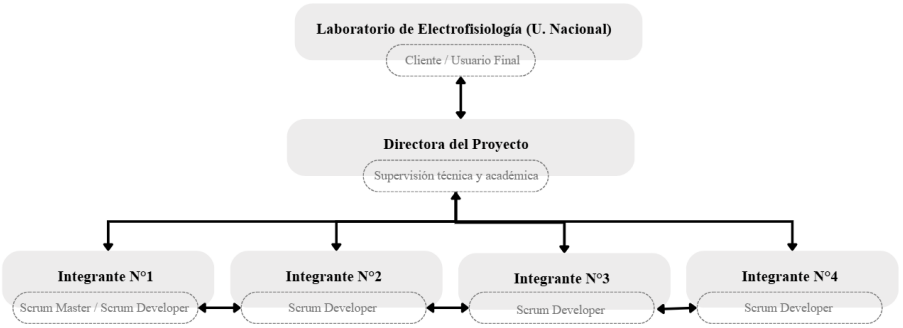


Figura 2: Organigrama y Flujo de Comunicación del proyecto GammaLab 2.0

4.5. Entregables del Proyecto

Como resultado del desarrollo de GammaLab 2.0 bajo el enfoque metodológico definido, el proyecto contempla la generación de entregables técnicos, funcionales y documentales asociados a las distintas fases del ciclo de vida. Estos productos permiten evidenciar el cumplimiento progresivo de los objetivos específicos y consolidan tanto los resultados internos del desarrollo como los

artefactos finales del proyecto.

VFP – Versión Final de Proyecto. Documento inicial que formaliza el problema, los objetivos, el alcance funcional y los lineamientos estratégicos y metodológicos del proyecto, sirviendo como base de referencia para la construcción del PMP y de los documentos técnicos posteriores.

PMP – Plan de Administración del Proyecto. Define los procesos de dirección del proyecto, la planificación y los mecanismos de seguimiento, estableciendo la estructura organizacional, los roles y el cronograma de trabajo.

MRCU – Matriz de Requerimientos y Casos de Uso Consolida la matriz original de requerimientos y casos de uso levantados durante la fase inicial del proyecto, registrando los históricos, conceptos, actores, requerimientos y cada caso de uso con su flujo principal y sus flujos alternos.

SRS – Especificación de Requerimientos de Software. Describe formalmente los requerimientos funcionales y no funcionales del sistema, formalizando las entradas, salidas, parámetros configurables y dependencias de cada módulo, incluyendo el nuevo módulo de análisis incorporado en GammaLab 2.0.

SAD – Documento de Arquitectura de Software. Formaliza las decisiones estructurales del sistema, incluyendo la distribución de componentes, las vistas arquitectónicas y la justificación técnica del diseño adoptado para GammaLab 2.0.

SDD – Documento de Diseño de Software. Presenta el diseño detallado de los componentes del sistema mediante diagramas de secuencia, diagramas de clases y demás representaciones estructurales, describiendo la organización modular y el comportamiento interno de GammaLab 2.0.

Mockups. Incluye las representaciones visuales de alta fidelidad de la interfaz de GammaLab 2.0, en las que se plasman gráficamente las mejoras planteadas en la experiencia de usuario y la navegación intuitiva del sistema, utilizadas para validar la disposición general de los módulos y los flujos de navegación.

Repositorio Final. Código fuente estructurado, versionado y documentado para facilitar mantenimiento, trazabilidad y futuras extensiones del proyecto.

Informe de Pruebas y Validación. Documentan resultados de pruebas técnicas, análisis comparativo entre GammaLab 1.0 y 2.0, y validación funcional de nuevas capacidades.

Manual de uso de GammaLab 2.0. Guía orientada al usuario final que describe instalación, navegación y uso de funcionalidades principales del sistema.

Memoria Final del Proyecto. Consolida el proceso completo de desarrollo, incluyendo contexto, metodología, diseño, implementación, pruebas, resultados y conclusiones.

4.6. Estructura colaborativa y enfoque de trabajo

Participante	**Rol**	**Descripción del rol**
Laboratorio de Electrofisiología (U. Nacional)	Cliente / Entidad beneficiaria	Define necesidades funcionales, valida resultados y recibe la entrega final del proyecto.
Cécile Gauthier Umaña	Directora del proyecto	Puente de comunicación con el cliente, orienta decisiones estratégicas y supervisa el desarrollo técnico y académico.
Juan Felipe Romero	Scrum Master	Asume funciones de liderazgo y coordinación general del equipo, manteniendo participación activa en el desarrollo técnico del proyecto.
Juan Felipe Romero, Laura Juliana Garzón, Sebastian Almanza y Xiomara Herrera	Scrum Developers	Participan en actividades de diseño, implementación, integración, pruebas y documentación del sistema.

Cuadro 5: Distribución de roles del proyecto

El desarrollo de GammaLab 2.0 se ejecuta bajo una dinámica de trabajo colaborativa, en la cual los cuatro integrantes participan activamente en las distintas fases del proyecto y comparten responsabilidad sobre el cumplimiento de los objetivos técnicos definidos. Aunque cada integrante asume liderazgo temporal sobre componentes específicos, el trabajo no se encuentra segmentado de manera rígida, sino que se favorece una distribución flexible de tareas según las necesidades de cada fase, el avance del proyecto y la complejidad técnica de los módulos involucrados. Esta estructura permite aprovechar las fortalezas individuales del equipo sin comprometer una visión integral del sistema.

Dentro de esta dinámica, el líder del equipo asume adicionalmente funciones de coordinación equivalentes al rol de *Scrum Master*. Paralelamente, se promueve la revisión cruzada, validación conjunta de decisiones técnicas y apoyo colaborativo en procesos de integración, pruebas y resolución de bloqueos.

Este esquema de trabajo responde directamente al enfoque híbrido Water-Scrum-Fall adoptado, permitiendo mantener una estructura organizada por fases

mientras se incorporan ciclos iterativos de trabajo y retroalimentación continua. Dicha retroalimentación puede provenir tanto del cliente, como principal fuente de validación funcional y requerimientos, como de la directora del proyecto, quien complementa este proceso mediante supervisión técnica y académica.

5. Administración del Proyecto

En esta sección se documentan los mecanismos de administración del proyecto GAMMA LAB 2.0, incluyendo estimación de actividades, inicio del proyecto, planes de trabajo, descomposición de actividades (WBS), calendarización, asignación de recursos y gestión de métricas.

5.1. Métodos y Herramientas de Estimación

Para la estimación de actividades del proyecto GAMMA LAB 2.0, se adopta un enfoque de estimación *Bottom-Up*, el cual parte de la identificación de tareas específicas y granulares, estima el esfuerzo requerido para cada una de ellas, y posteriormente agrega estas estimaciones para obtener el tiempo total del proyecto. Esta metodología es particularmente adecuada para proyectos en evolución donde se cuenta con experiencia de una versión anterior y donde las dependencias técnicas son claras.

El proceso inició con la definición de las cuatro fases del proyecto, correlacionadas directamente a los cuatro objetivos específicos:

- **Fase 1: Identificación de brechas de mejora en GammaLab 1.0** – 3 semanas
- **Fase 2: Diseño del planteamiento estructural de la solución** – 4 semanas
- **Fase 3: Implementación técnica** – 7 semanas
- **Fase 4: Validación y pruebas con el usuario** – 2 semanas

A partir de esta estructura, el equipo identificó las actividades concretas dentro de cada fase, desagregó estas actividades en tareas específicas y estimó el esfuerzo en horas-persona requerido para cada tarea. Esta información se consolidó en una tabla de estimación detallada que refleja la carga de trabajo esperada por fase.

5.1.1. Supuestos de Estimación

La estimación se fundamenta en los siguientes supuestos:

- El equipo de desarrollo está compuesto por cuatro desarrolladores.
- Cada desarrollador dispone de un mínimo de 6 horas semanales de disponibilidad efectiva para trabajo de desarrollo, análisis, diseño, codificación y pruebas.
- La jornada semanal de dedicación será distribuida de manera flexible entre las actividades, con énfasis en mantener sincronización a través de canales de comunicación asíncrona (GitHub, comunicación escrita).
- Se reutilizará conocimiento y código de GAMMA LAB 1.0, reduciendo el tiempo de aprendizaje inicial.
- La directora del proyecto está disponible para sesiones de revisión semanal y proporciona retroalimentación en plazos acordados.
- El cliente (Laboratorio de Electrofisiología) participa activamente en validaciones de línea base (Fase 1) y en pruebas finales (Fase 4).
- No existen restricciones significativas de recursos técnicos o de infraestructura que impidan la ejecución del proyecto.
- Las metodologías de desarrollo (Water-Scrum-Fall) y las herramientas (Git, el Kanban del departamento, Python, PyQt5) permanecen estables durante la ejecución.

5.1.2. Estimación de Tiempo por Fases del Proyecto

La siguiente tabla resume la estimación de duración por fases, con base en el análisis Bottom-Up de actividades y tareas:

Fase	**Descripción**	**Duración**	**Inicio**	**Cierre**
1	Identificación de brechas	3 sem	27 jul	16 ago
2	Diseño de solución	4 sem	19 ago	13 sep
3	Implementación técnica	7 sem	16 sep	27 oct
4	Validación y pruebas	2 sem	28 oct	20 nov
Total estimado		**16 sem**	**27 jul**	**20 nov**

Cuadro 6: Resumen de calendarización del proyecto GAMMA LAB 2.0

5.1.3. Distribución de Esfuerzo por Fase

Basado en el análisis Bottom-Up de tareas, la siguiente tabla detalla el esfuerzo estimado (en horas-persona) para cada fase:

Fase	Duración (sem)	Esfuerzo Total (h-persona)
Fase 1: Identificación de brechas	3	72 h
Fase 2: Diseño de solución	4	96 h
Fase 3: Implementación técnica	7	168 h
Fase 4: Validación y pruebas	2	48 h
Total	**16**	**384 h**

Cuadro 7: Estimación de esfuerzo por fases del proyecto (Bottom-Up)

Nota: El esfuerzo total de 384 horas se distribuye entre 4 desarrolladores, resultando en 96 horas por persona en las 16 semanas, equivalentes a 6 horas semanales promedio por desarrollador.

5.2. Inicio del Proyecto

5.2.1. Entrenamiento del Personal

El equipo de GAMMA LAB 2.0 está compuesto por cuatro desarrolladores nuevos, por lo que es fundamental un plan estructurado de transferencia de conocimiento y entrenamiento que les permita:

- Comprender la arquitectura y funcionamiento de GAMMA LAB 1.0
- Familiarizarse con las tecnologías base (Python, PyQt5, VTK, etc.)
- Alinearse con los estándares de código y de documentación del proyecto
- Entender el dominio técnico (procesamiento de señales EEG, análisis en dominios de tiempo, frecuencia y tiempo-frecuencia)

El plan de entrenamiento se ejecutará durante la Fase 1 (primeras 3 semanas) de manera paralela a las actividades de análisis de brechas. Las actividades incluyen:

1. **Sesiones de transferencia con equipo de GAMMA LAB 1.0:** Reuniones semiestructuradas donde el equipo original explica decisiones de diseño, limitaciones técnicas y funcionalidades planificadas pero no implementadas.
2. **Exploración técnica autónoma:** Cada desarrollador realiza una revisión independiente del código fuente, arquitectura y documentación de

GAMMA LAB 1.0, generando notas y preguntas que se resuelven en sesiones grupales.

3. **Capacitación en dominio (EEG y procesamiento de señales):** Sesiones cortas sobre conceptos clave de electroencefalografía, análisis en dominios de tiempo, frecuencia y tiempo-frecuencia, así como sobre las nuevas técnicas que se implementarán (amplitude coupling, paralelización).
4. **Alineación en estándares y herramientas:** Revisión de estándares de código (PEP8), políticas de control de versiones (Git, Pull Requests), estructura de repositorio y convenciones de documentación.
5. **Sesiones de lluvia de ideas colaborativa:** El equipo completo (desarrolladores, directora, y ocasionalmente el cliente) participa en sesiones para alinear la visión del proyecto y clarificar alcances y expectativas.

5.2.2. Infraestructura

Para el desarrollo de GAMMA LAB 2.0, se dispone de la siguiente infraestructura técnica y de colaboración:

Infraestructura física:

- Cada desarrollador cuenta con su propio equipo de desarrollo (laptop o computadora de escritorio) con capacidad suficiente para programación, ejecución de código Python y pruebas locales.
- Se solicita acceso a una máquina virtual compartida que actúe como entorno de integración centralizado, permitiendo verificar compatibilidad multiplataforma y ejecutar pruebas de integración antes de la entrega final.

Infraestructura lógica y de colaboración:

- **Control de versiones:** Git con repositorio centralizado en GitHub. La estructura del repositorio se basa en ramas por frente de trabajo (UI/UX, paralelización núcleo, amplitude coupling, paralelización módulo nuevo), con política de Pull Requests obligatoria para integración.
- **Entorno de desarrollo:** Visual Studio Code, PyCharm o editor de preferencia de cada desarrollador, con extensiones para linting (PEP8), debugging y testing.
- **Gestión del proyecto:** tablero Kanban proporcionado por el departamento, con columnas: “To Do”, “In Progress”, “In Review”, “Done”. Sincronización asíncrona diaria a través de comentarios en tarjetas de tareas.
- **Comunicación:** Microsoft Teams para reuniones semanales formales con la directora; WhatsApp para coordinación ágil y resolución rápida de dudas.

- **Documentación técnica:** Overleaf para memoria del proyecto en LaTeX; Microsoft Word como herramienta auxiliar para redacción preliminar y colaboración paralela.
- **Prototipado y diseño:** Figma para diseño visual de la interfaz de usuario, con acceso compartido para revisión y validación iterativa.
- **Pruebas automatizadas:** Pytest como framework para pruebas unitarias, con configuración para ejecución local en cada desarrollo y en la máquina virtual compartida.
- **Respaldo y almacenamiento:** OneDrive para documentos y respaldo incremental; GitHub como respaldo de código fuente. Se ejecutará una política semanal de respaldo completo de código, documentación y artefactos del proyecto.

5.3. Asignación de Recursos

5.3.1. Equipo de Desarrollo

El equipo de GAMMA LAB 2.0 está conformado por cuatro estudiantes de Ingeniería de Sistemas de la Pontificia Universidad Javeriana, todos dedicados al desarrollo del proyecto durante las 16 semanas de ejecución.

Nombre	**Rol**	**Responsabilidad**
Juan Felipe Romero Ortiz	Desarrollador + Scrum Master	Frente 1 (UX/UI)
Laura Juliana Garzón Arias	Desarrollador	Frente 2 (Paralelización)
Xiomara Herrera Suárez	Desarrollador	Frente 3 (Amplitude Coupling)
Sebastian Almanza Galvis	Desarrollador	Frente 4 (Paralelización módulo)

Cuadro 8: Equipo de desarrollo de GAMMA LAB 2.0

Notas sobre asignación de roles:

- El rol de Scrum Master es asumido por Juan Felipe Romero Ortiz sin que esto implique jerarquía en la toma de decisiones. Sus responsabilidades incluyen agendar reuniones, dar seguimiento al progreso visible en el Kanban del departamento, mantener el enfoque del equipo y facilitar la comunicación con la directora.
- La distribución de tareas en cada sprint se define colectivamente mediante consenso del equipo, considerando las especialidades, la carga actual y las dependencias entre frentes de trabajo.

- Cada desarrollador es responsable de código de calidad (según PEP8), de incluir pruebas unitarias para sus cambios, y de participar activamente en revisión de código de sus compañeros.
- La asignación a frentes de trabajo no es exclusiva; los desarrolladores colaborarán entre frentes según las necesidades de cada sprint.

5.3.2. Disponibilidad Horaria por Desarrollador

Cada desarrollador dispone de un mínimo de 6 horas semanales de dedicación efectiva al proyecto. Esta disponibilidad se distribuye de la siguiente manera:

Desarrollador	**Horas/sem**	**Total (16 sem)**	**Promedio por fase**
Juan Felipe Romero	6	96 h	24-28 h
Laura Juliana Garzón	6	96 h	24-28 h
Xiomara Herrera	6	96 h	24-28 h
Sebastian Almanza	6	96 h	24-28 h
Total equipo	**24**	**384 h**	**96-112 h**

Cuadro 9: Distribución de disponibilidad horaria por desarrollador

Gestión de disponibilidad:

- La dedicación es flexible en términos de distribución semanal, permitiendo que algunos desarrolladores trabajen más horas en semanas con dependencias críticas y menos en semanas de menor carga.
- Se espera que los desarrolladores comuniquen anticipadamente cualquier conflicto o reducción temporal en disponibilidad, para permitir que el equipo se reorganice.
- Las reuniones formales (Sprint Planning, Sprint Review, Sprint Retrospective, reuniones con directora) son obligatorias y están fuera de las 6 horas semanales de desarrollo estimadas. Se espera que no superen 2 horas semanales en total.
- La comunicación asíncrona a través de GitHub Issues, Pull Request comments y WhatsApp reduce la necesidad de reuniones en tiempo real, permitiendo máxima flexibilidad horaria.

5.3.3. Directora del Proyecto

Nombre	Rol	Dedicación
Cecile Eugenie Gloria Gauthier Umaña	Directora + Product Owner	10 %

Cuadro 10: Directora del proyecto

Responsabilidades de la directora:

- Actúa como Product Owner, representando los intereses del cliente (Laboratorio de Electrofisiología) y priorizando el Product Backlog.
- Participa en Sprint Planning y Sprint Review semanales para alineación y validación.
- Proporciona retroalimentación sobre artefactos de diseño, código y documentación en plazos acordados (máximo 48 horas).
- Facilita comunicación entre el equipo de desarrollo y el cliente, recogiendo feedback y necesidades emergentes.
- Asegura el cumplimiento de estándares de calidad y de alineación con los objetivos planteados.

5.3.4. Cliente

Entidad	Rol	Participación
Laboratorio de Electrofisiología - Universidad Nacional de Colombia	Usuario final + Validador	Fases 1, 2, 4

Cuadro 11: Cliente del proyecto

Puntos de contacto con el cliente:

- **Fase 1:** Sesión formal de levantamiento de requerimientos, ejecución de pruebas de línea base sobre GAMMA LAB 1.0.
- **Fase 2:** Validación de prototipo de interfaz de usuario y especificaciones técnicas del nuevo módulo.
- **Fase 4:** Participación en pruebas funcionales, validación de amplitude coupling, pruebas de usabilidad y firma de aceptación final.

5.4. Gestión de Cambios y Control de Alcance

5.4.1. Solicitudes de Cambio

Durante el desarrollo, pueden surgir solicitudes de cambio de requerimientos, modificaciones de diseño o ajustes de alcance. Estas se manejan mediante el siguiente protocolo:

1. **Identificación:** El cambio es identificado por cualquier miembro del equipo o del cliente.
2. **Documentación:** Se crea una GitHub Issue con descripción clara del cambio, justificación e impacto estimado.
3. **Evaluación:** El equipo y la directora evalúan el impacto en cronograma, recursos y alcance.
4. **Priorización:** El cambio se añade al Product Backlog con prioridad asignada por consenso.
5. **Integración:** El cambio se integra en el sprint siguiente si es de alta prioridad, o se pospone si afecta significativamente el cronograma.

Cambios solicitados después de la mitad de Fase 3 (después del sprint 7) serán evaluados para versiones futuras, priorizando la entrega a tiempo de los objetivos planteados.

5.5. Criterios de Éxito del Proyecto

El proyecto GAMMA LAB 2.0 será considerado exitoso cuando se cumplan los siguientes criterios cuantificables:

1. **Rendimiento mejorado:** GAMMA LAB 2.0 debe procesar señales EEG significativamente más rápido que GAMMA LAB 1.0. Se espera una mejora de al menos 20-30 % en tiempos de procesamiento derivada de la paralelización del sistema.
2. **Nuevas funcionalidades correctas al 100 %:** El módulo de amplitude coupling debe operar sin errores críticos y producir resultados reproducibles y validables. Las pruebas unitarias deben pasar al 100 %.
3. **Corrección de problemas de v1.0:** Los errores identificados en GAMMA LAB 1.0 que fueron priorizados en el Product Backlog (Fase 1) deben ser corregidos y validados por el cliente.
4. **Validación de usuario:** El usuario final (Laboratorio de Electrofisiología) debe confirmar en la Fase 4 que:

- Las nuevas funcionalidades responden a sus necesidades.
- El sistema es intuitivo y fácil de usar.
- Los resultados del análisis son precisos y confiables.
- La documentación y manuales son claros.

5. **Cobertura de pruebas:** Mínimo 80 % cobertura de código en pruebas unitarias. Todo código nuevo debe incluir casos de prueba.
6. **Documentación completa:** Memoria del proyecto con todas las secciones requeridas, manuales técnico y de usuario, comentarios en código y docstrings en funciones Python.
7. **Entrega a tiempo:** El proyecto debe cerrar el 20 de noviembre de 2026 con todos los entregables completados según este PMP.

5.6. Monitoreo y Métricas de Avance

5.6.1. Métrica Clave: Velocidad del Equipo

La *velocidad* se define como el número de horas-persona completadas (según Definition of Done) por sprint. Esto permite:

- Proyectar si el equipo completará el proyecto a tiempo.
- Identificar cuellos de botella u obstáculos.
- Ajustar la distribución de trabajo en sprints futuros.

Seguimiento de velocidad:

Métrica	**Objetivo**	**Frecuencia**	**Responsable**
Velocidad por sprint	24 h/semana	Semanal	Scrum Master
Burn-down chart	Progresión lineal	Semanal	Scrum Master
Definition of Done	100 % cumplimiento	Por ítem	Equipo
Tasa de defectos	Minimizar	Por sprint	Testing

Cuadro 12: Métricas clave de monitoreo

5.6.2. Reuniones de Sincronización

- **Sprint Planning:** Lunes de cada semana, 30 minutos. Selección de tareas del backlog y distribución entre desarrolladores.
- **Sincronización asíncrona:** Diariamente a través de comentarios en GitHub Issues. Cada desarrollador actualiza el estado de sus tareas.

- **Sprint Review:** Viernes de cada semana, 45 minutos. Presentación de incrementos completados a la directora. Retroalimentación.
- **Sprint Retrospective:** Viernes de cada semana (después de Review), 30 minutos. Análisis interno del equipo sobre qué funcionó bien, qué mejorar, y acciones correctivas para próximo sprint.
- **Sesiones técnicas:** Según necesidad (no planificadas regularmente). Para resolver decisiones arquitectónicas, resolver bloqueos o validaciones cruzadas entre frentes.

6. Monitoreo y Control del Proyecto

6.1. Administración de Requerimientos

Descripción del proceso

La administración de requerimientos en GammaLab 2.0 se concibe como un proceso iterativo y controlado que permite gestionar los cambios de forma ordenada y trazable a lo largo del ciclo de vida del proyecto bajo el enfoque Water-Scrum-Fall. Este proceso involucra tanto al equipo de desarrollo con responsabilidades claras para la identificación, documentación, revisión y validación de los requerimientos funcionales y no funcionales del sistema.

La gestión de requerimientos comienza en la Fase 1, donde se identifican brechas de mejora a través de exploración técnica de GammaLab 1.0, entrevistas semiestructuradas con el equipo original y sesiones formales de levantamiento de requerimientos con el cliente. Posteriormente, estos requerimientos se modelan en casos de uso, se priorizan en el Product Backlog y se documentan formalmente. Cada requerimiento se asocia a una matriz de trazabilidad que permite vincularlo con componentes del diseño, código, pruebas y entregables.

Durante el desarrollo iterativo (sprints), los requerimientos pueden ser refinados con base a una constante retroalimentación. El proceso de control incluye:

- Un canal formal de solicitud de cambio.
- Evaluación del impacto técnico y temporal sobre el cronograma.
- Aprobación del cambio por consenso del equipo y la directora del proyecto.
- Actualización de todos los artefactos afectados: Product Backlog, documentos, matriz de trazabilidad, prototipos y código.

Los cambios solicitados después de la mitad de la Fase 3 (sprint 7) serán evaluados para versiones futuras, priorizando la entrega a tiempo de los objetivos planteados.

Artefactos por generar

Durante la administración de requerimientos se generarán los siguientes artefactos:

6.2. Monitoreo y Control de Progreso

El seguimiento del progreso del proyecto se realizará mediante un sistema combinado de control visual, medición de métricas clave y reportes periódicos. El

Artefacto	Descripción
Product Backlog inicial	Lista priorizada de brechas y requerimientos identificados, acordados con el cliente al cierre de la Fase 1.
Casos de uso	Diagramas y descripciones narrativas que modelan las interacciones del usuario con el sistema para cada frente de trabajo.
Matriz de trazabilidad	Tabla que relaciona requerimientos con elementos de diseño, código y pruebas para garantizar cobertura completa.
Registro de solicitudes de cambio	documentación cada solicitud de cambio con su justificación, impacto estimado y estado de resolución.
Historial de versiones	Registro en GitHub que documenta los cambios aceptados en el repositorio y su justificación técnica.

Cuadro 13: Artefactos generados durante la administración de requerimientos

objetivo de este monitoreo es asegurar que el desarrollo se mantenga alineado con los tiempos, entregables y objetivos definidos en la planificación bajo el enfoque Water-Scrum-Fall.

Para ello, se implementarán tres estrategias complementarias:

- **Diagrama de Gantt:** permite observar la duración y el solapamiento de actividades, así como su estado de avance en relación con la línea temporal del proyecto.
- **Burnup Chart en GitHub:** herramienta gráfica que compara el avance real frente al avance planificado en horas-persona, útil para identificar retrasos o desviaciones respecto a la velocidad objetivo del equipo.
- **Tablero Kanban :** Muestra el estado del trabajo (*To Do*, *In Progress*, *In Review*, *Done*), facilitando la identificación de bloqueos y la sincronización asíncrona diaria del equipo.

Actividad	Responsable	Frecuencia	Herramienta
Actualización del tablero Kanban	Todo el equipo	Diaria	Kanban del departamento
Verificación de velocidad del sprint	Scrum Master	Semanal	Kanban del departamento
Registro de desvíos y acciones correctivas	Scrum Master	Semanal	Hoja de seguimiento
Sprint Review con la directora	Equipo y Directora	Semanal (viernes)	Burnup Chart
Revisión del avance por fase	Directora del proyecto	Al cierre de fase	Gantt + Checklist

Cuadro 14: Actividades de monitoreo y responsables

6.3. Cierre del Proyecto

El cierre del proyecto contempla una serie de actividades orientadas a garantizar la entrega formal, validación y documentación del producto final. Las actividades clave serán:

- **Sprint Review final:** presentación de los resultados comparativos entre GammaLab 1.0 y 2.0, y validación de las funcionalidades nuevas con la directora del proyecto.
- **Evaluación post-mortem del ciclo:** reflexión grupal sobre aprendizajes, errores y oportunidades de mejora para versiones futuras.
- **Revisión final de entregables:** verificación de completitud, actualización y correcta documentación de todos los artefactos del proyecto.
- **Prueba final del sistema:** ejecutada en condiciones reales de uso con investigadores del Laboratorio de Electrofisiología de la Universidad Nacional de Colombia.
- **Entrega formal del software:** incluye instalador, código fuente organizado, manual técnico y manual de usuario de GammaLab 2.0.
- **Firma de aceptación del cliente:** acta de conformidad posterior a la validación del sistema por parte del laboratorio.
- **Consolidación de documentación:** almacenamiento definitivo de todos los entregables en OneDrive y GitHub con tags de versión final.

6.4. Entrega del Producto

La entrega de GammaLab 2.0 alcliente se realizará durante la semana 17 del calendario académico Javeriana 2026-30, correspondiente al cierre de la Fase 4 según el calendario del proyecto. Se entregará el código fuente, los manuales técnicos y las guías de usuario. El equipo de desarrollo será responsable de realizar la instalación del sistema en los equipos del Laboratorio de Electrofisiología que cuenten con sistema operativo compatible.

Una vez instalado, se ejecutará una prueba funcional con datos reales del laboratorio. Se evaluará también la usabilidad del sistema mediante sesiones guiadas y no guiadas con los investigadores, contrastando los resultados frente a la línea base establecida en la Fase 1.

Referencias

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.

[2] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org, 2020. [Online]. Available: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf

[3] I. Sommerville, *Software Engineering*, 10th ed. Harlow, England: Pearson Education, 2016.

[4] N. Yahya and S. S. Maidin, "The waterfall model with agile Scrum as the hybrid agile model for the software engineering team," in *2022 10th International Conference on Cyber and IT Service Management (CITSM)*, 2022, pp. 1–6, doi: 10.1109/CITSM56380.2022.9936036.

[5] Academia.edu, "Comparative study of various process model in software development," 2012. [Online]. Available: https://www.academia.edu/81531661